Technische Hochschule Nürnberg

Machine Learning

Die lineare Regression

Prof. Dr. Tobias Bocklet

Regression – ALVINN



https://www.youtube.com/watch?v=2KMAAmkz9go (ab ca. Minute 2)

Beispiel überwachtes Lernen – ALVINN

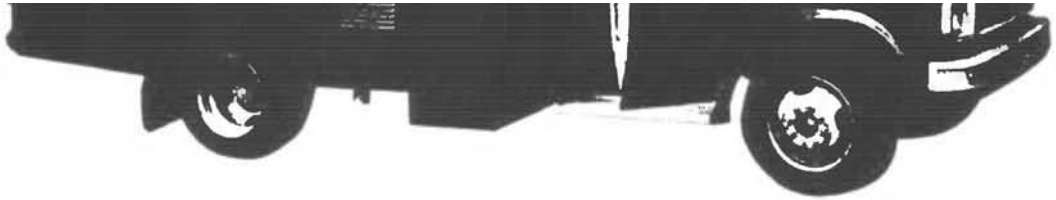
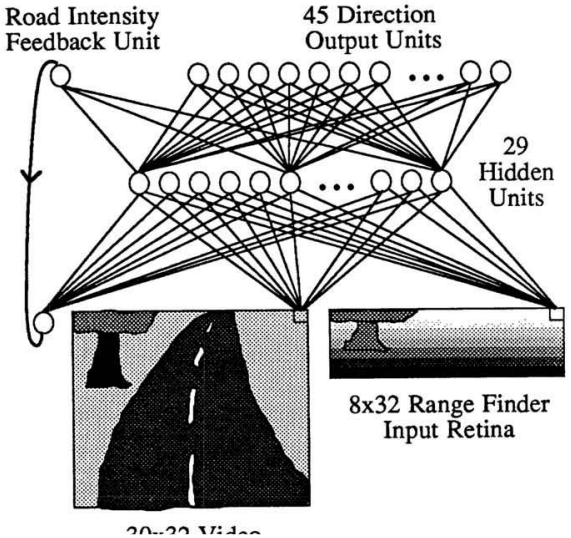


Figure 3: NAVLAB, the CMU autonomous navigation test vehicle.

pagation simulator described in [Pomerleau et al., 1988]. Training involves first cre
g a set of 1200 road snapshots depicting roads with a wide variety of retinal orienta
s and positions, under a variety of lighting conditions and with realistic noise levels
k-propagation is then conducted using this set of exemplars until only asymptoti
formance improvements appear likely. During the early stages of training, the inpu
l intensity unit is given a random activation level. This is done to prevent the net
k from merely learning to copy the activation level of the input road intensity uni
he output road intensity unit, since their activation levels should almost always b
ıtical because the relative intensity of the road and the non-road does not often chang

eau



Heute – Lineare Regression

- Regressionsproblem
- Wie trainiere ich eine Regression?
  - Analytisch
  - Iterativ (Dienstag)
- Gütebestimmung & Evaluation

Regressionsproblem

- Datensatz mit Merkmalen $\mathbf{x} = (x_1, x_2, \ldots, x_d)$
- Regression: Lernen der Abbildung $f(\mathbf{x}) : \mathbb{R}^d \mapsto \mathbb{R}$
- Zum Vergleich Klassifikation: $f(\mathbf{x}) : \mathbb{R}^d \mapsto \mathcal{Y}$

$$\mathcal{Y} = \{1, 2, \ldots, C\}$$

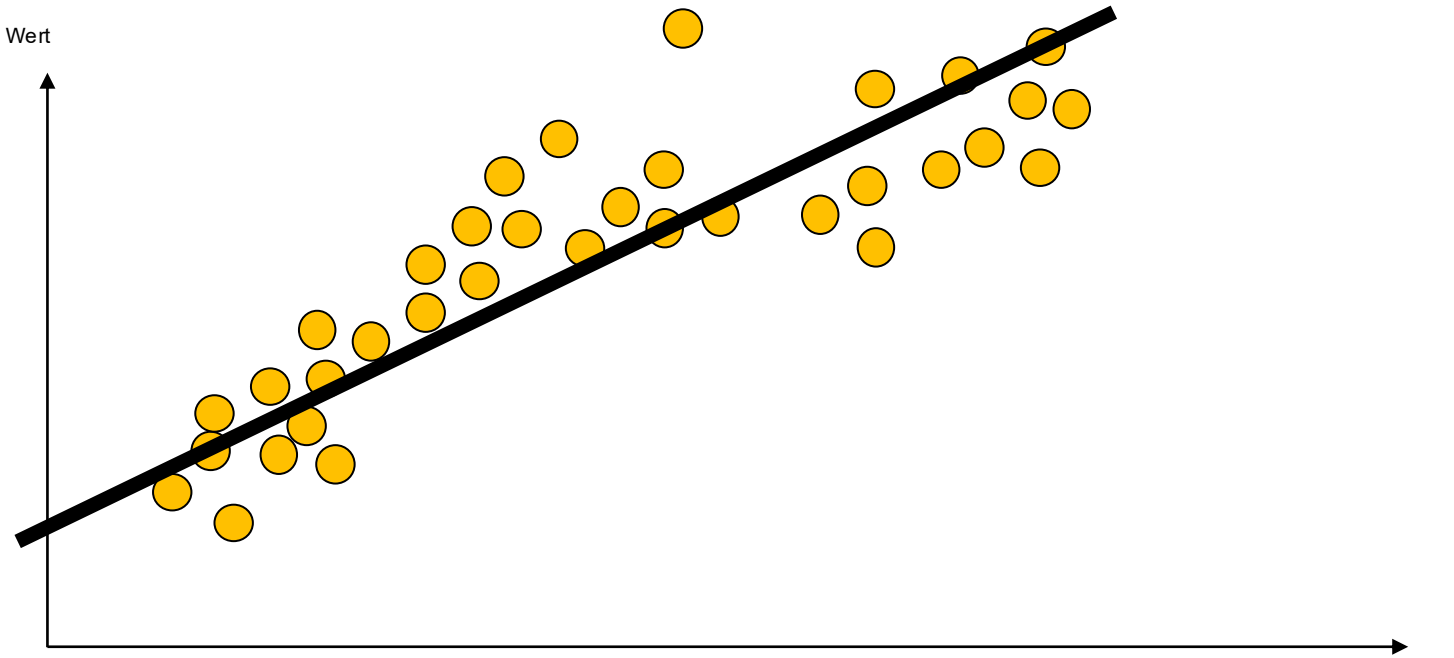
Regressionsanalyse

- Statistisches Analyseverfahren mit dem Ziel Beziehungen zwischen einer abhängigen und einer oder mehreren unabhängigen Variablen zu modellieren.
- Anhängige Variable = Zielwert y
- Unabhängige Variablen = Merkmale **x**
- Bestimmen eines funktionalen Zusammenhangs zwischen **x** und y

$$\hat{y} = f(\boldsymbol{x})$$

- Praktische Anwendungen:
  - Vorhersage: Merkmale **x** liegen vor, sage Zielwert y vorher
  - Korrelation: Quantifizieren der Stärke des Zusammenhangs zwischen $x_{\mathrm{i}}, \ldots, \mathrm{x_D}$ und y

Regression



Trainingsbeispiele Merkmal 1 mit Wert

Ziel: Schätzen einer Funktion

Schätzen von Regressionsmodellen

- Form der Funktion $f(x)$ muss festgelegt werden (z.B. Lineare Regression)

$$f(x) = mx + b$$

- Datensammlung $(\mathbf{X}, \boldsymbol{y})$ → Menge von M Vektoren mit zugehörigen Zielwerten
  - Genügend Daten um die Parameter von $f(x)$ zu schätzen
  - Kann als ein überbestimmtes Gleichungssystem angesehen werden

Regression

- Überwachtes Lernen
- Finde Gerade $f(x)$, die möglichst nahe zu den Datenpunkten ist
- Fehler: Vorhergesagter Wert != Tatsächlicher Wert
- Kostenfunktion: Wie weit liegt tatsächlicher Wert $\hat{y}$ vom vorhergesagten Wert $f(x)$ entfernt

$$C = \frac{1}{2M} \sum_{i=1}^{M} \left(y_i - f(x_i)\right)^2$$

- Ziel: Minimieren der Kostenfunktion

Wie lösen?

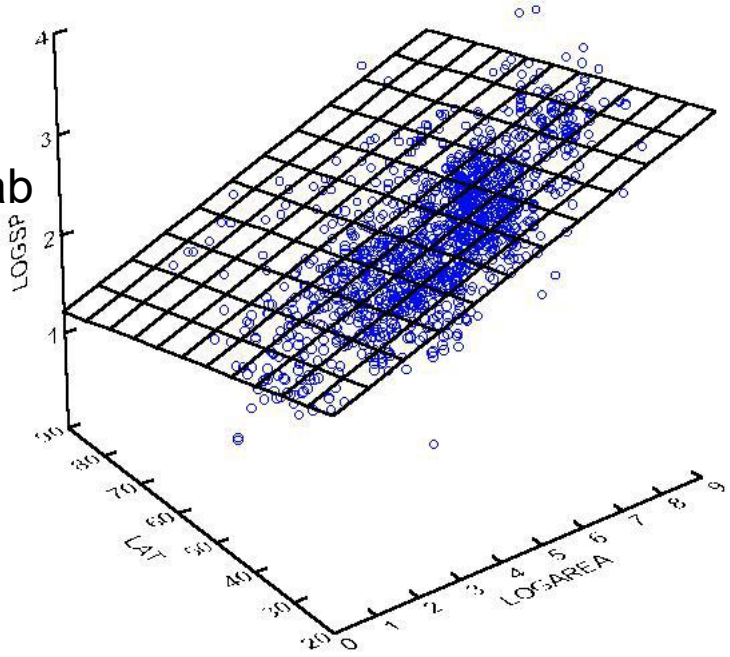
- Analytisch mittels Ordinary Least Squares (OLS)

  (Tafel)

- Iterativ mittels Gradientenabstieg

Multivariate Lineare Regression

http://ordination.okstate.edu/MULTIPLE.htm



- Auch Mehrfachregression genannt
- Zielwert y hängt von mehreren Werten $x_1, \dots, x_D$ ab
- Beispiel M Datenpunkte:

$$y_1 = w_1 * x_{11} + w_2 * x_{12} + \dots + w_D * x_{1D} + b$$

$$y_2 = w_1 * x_{21} + w_2 * x_{22} + \dots + w_D * x_{2D} + b$$

$$\vdots$$

$$y_M = w_1 * x_{M1} + w_2 * x_{M2} + \dots + w_D * x_{MD} + b$$

Geschrieben als Vektoren und Matrizen:

$$\begin{pmatrix} y_1 \\ y_2 \\ \vdots \\ y_M \end{pmatrix} \begin{pmatrix} w_1 \\ w_2 \\ \vdots \\ w_D \end{pmatrix} \begin{pmatrix} b_1 \\ b_2 \\ \vdots \\ b_M \end{pmatrix} \quad X = \begin{pmatrix} x_{11} & x_{12} & \cdots & x_{1D} \\ x_{21} & x_{22} & \cdots & x_{2D} \\ \vdots & \vdots & \ddots & \vdots \\ x_{M1} & x_{M2} & \dots & x_{MD} \end{pmatrix}$$

$$\boldsymbol{y = w^T X + b}$$

OLS bei multivariater Linearer Regression

$$C = \frac{1}{2M} \sum_{i=1}^{M} (f(x_i) - y_i)^2$$

$$\mathrm{C} = (\mathrm{Xw} - \mathrm{y})^{\mathrm{T}} (\mathrm{Xw} - \mathrm{y})$$

$$\frac{\partial C}{\partial w} \Rightarrow X^T X\, w - X^T \hat{y} = 0$$

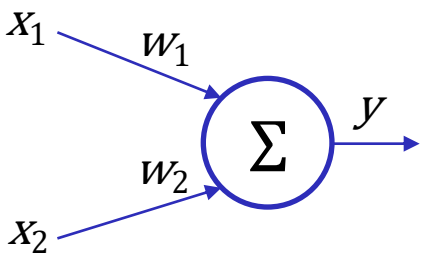
Heute – Lineare Regression

- Regressionsproblem
- Wie trainiere ich eine Regression?
  - Analytisch (Letzte Woche)
  - Iterativ
- Gütebestimmung & Evaluation

Iterative Lösung – Der Gradientenabstieg

- Generischer Optimierungsalgorithmus
- Findet (lokale) Minima
- Startet mit initialen Gewichten
- Passt diese iterativ an um Kosten zu minimieren

Gradientenabstieg



- Ausgabe y ist gewichtetet Summe der Eingaben x_i

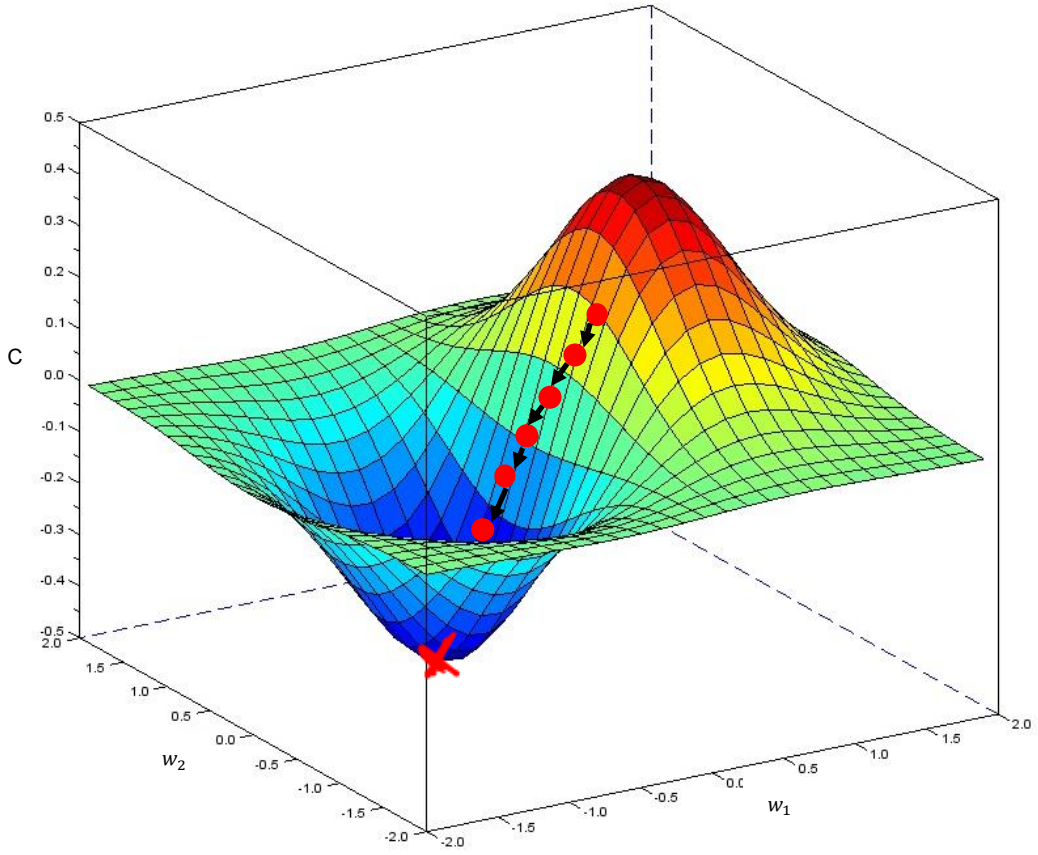
$$\hat{y} = \sum_i x_i w_i = \boldsymbol{w}^T \boldsymbol{x}$$

- Training erfordert Trainingsbeispiele mit bekannten Ausgaben $\hat{y}$
- Quadratische Fehler- bzw. Kostenfunktion:

$$C = \frac{1}{2}(\hat{y} - y)^2$$

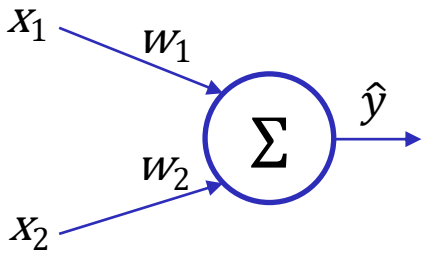
- Iteratives Verfahren zur Minimierung der Kostenfunktion C durch Anpassen der Gewichte $\boldsymbol{w}$

Gradientenabstieg



Iteratives Verfahren

- Iteratives Verfahren anhand eines Beispiels
  - Beginne mit zufälligen Gewichten $\boldsymbol{w}$
  - Bestimme y für jedes Element der Lernstichprobe



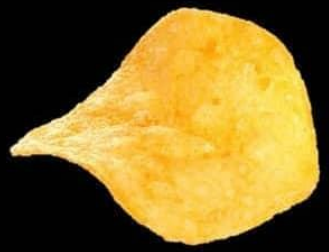
  - Anhand des Fehlers zum gewünschten Ergebnis y wird eine Korrektur der Gewichte durchgeführt

- Datengetrieben, numerisch


What others see
Potato chip

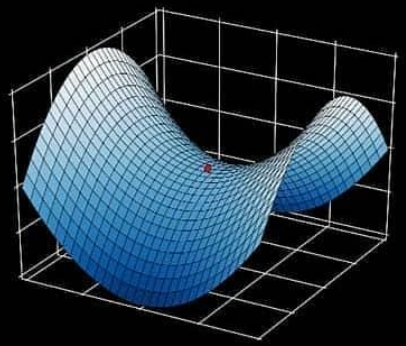
What others see

Potato chip





What i see



Gradient descent

Gradient descent is an iterative optimization algorithm for finding the **minimum** of a function.

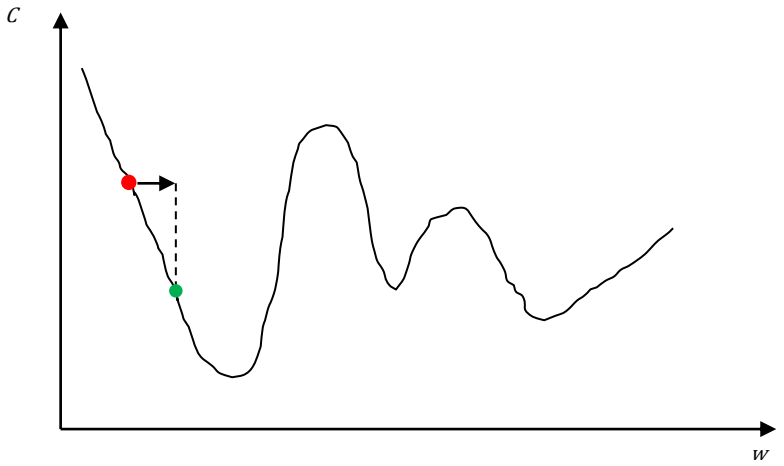
next position — *opposite direction* — *gradient at current position*

$$x_{i+1} = x_i - \alpha \nabla f(x_i)$$

current position — *step size (learning rate)*

Gradientenabstiegsverfahren

- Durch Änderung der Gewichte sollen die Kosten C minimiert werden
- Folge dem Gradienten um Minimum in C zu finden



- Änderung der Gewichte

$$w_i \rightarrow w_{i_{neu}} = w_i + \Delta w_i$$

mit $\Delta w_i = -\varepsilon \frac{\partial C}{\partial w_i}$

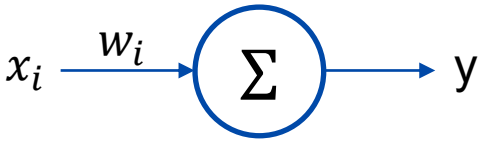
Wie kommen wir zur Delta-Regel

Durch Einsetzen unserer quadratischen Kostenfunktion:

$$C = \frac{1}{2}(\hat{y} - y)^2$$

Kettenregel:

$$\frac{\partial C}{\partial w_i} = \frac{dC}{dy} \frac{\partial y}{\partial w_i} = (-1)(\hat{y} - y)\, x_i$$



$$y = w_i x_i \qquad \frac{\partial y}{\partial w_i} = x_i$$

Delta-Regel:

$$\Delta w_i = -\varepsilon \frac{\partial C}{\partial w_i} = \varepsilon x_i (\hat{y} - y)$$

Pseudocode

- Eingabe
  - $\boldsymbol{X}$ **:** Matrix mit n Trainingsvektoren/Beispielen
  - Randomisiert initialisierte Werte für w_0 und w_1
  - predict(X): $\mathbf{y} = w_1 \boldsymbol{X} + w_0$
  - learnRate
  - # of epochs

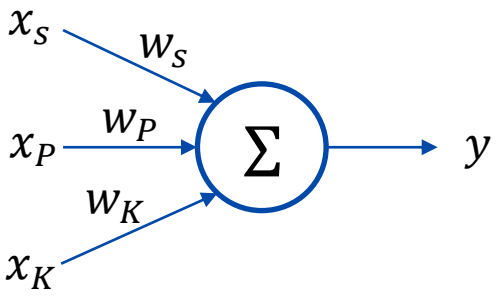
- Fit → For e in epochs:
  - $\mathbf{y_pred}$ = predict($\mathbf{X}$)
  - $delta_w_1$ = (-1/n) * sum($\boldsymbol{X} * (\mathbf{y} - \mathbf{y_pred})$) # Partielle Ableitung für w_1 → $\frac{\partial C}{\partial w_1}$
  - $delta_w_0$ = (-1/n) * sum($\mathbf{y} - \mathbf{y_pred}$) # Partielle Ableitung für w_0 → $\frac{\partial C}{\partial w_0}$
  - w_1 = w_1 - learnRate * $delta_w_1$ # Update w_1
  - w_0 = w_0 - learnRate * $delta_w_0$ # Update w_0

Beispiel: Mensa

- Sie gehen jeden Tag in die Mensa
  - → Sie essen jeden Tag Schnitzel mit Pommes und Ketchup
  - → Die Portionsgrößen variieren
- Der Kassierer nennt ihnen nur den Gesamtpreis
  - → Nach ein paar Tagen sollte man den Preis der Einheiten (=Gewichte) bestimmen können
- Iterativer Ansatz: Wir starten mit zufälligen Gewichten und passen diese an um den Fehler der Gesamtpreise zu minimieren

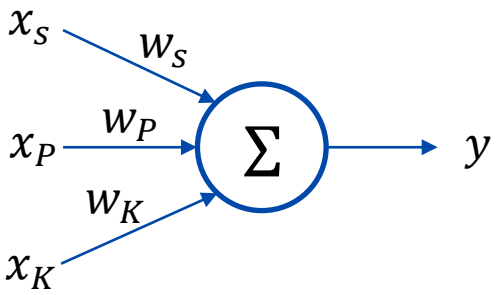
Beispiel: Mensa

- Eingabevektor $\boldsymbol{x} = \left(x_{schnitzel}, x_{Pommes}, x_{Ketchup}\right)$
- Gewichtsvektor $\boldsymbol{w} = \left(w_{schnitzel}, w_{Pommes}, w_{Ketchup}\right)$
- Ausgabe $y = x_S w_s + x_P w_p + x_K w_K$



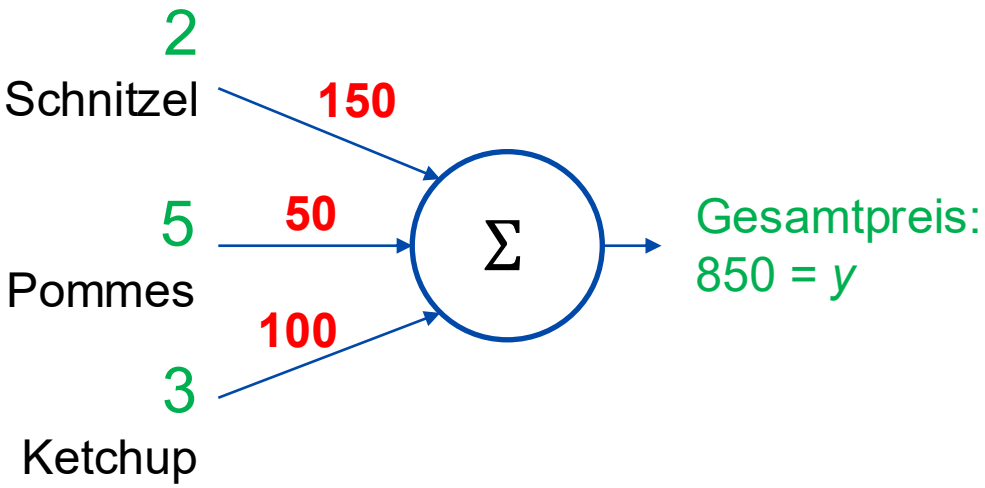
Beispiel: Mensa

- Eingabevektor $\boldsymbol{x} = (x_{schnitzel}, x_{Pommes}, x_{Ketchup})$ → Anzahl der Einheiten
- Gewichtsvektor $\boldsymbol{w} = (w_{schnitzel}, w_{Pommes}, w_{Ketchup})$ → Preis der Einheiten
- Ausgabe $y = x_S w_s + x_P w_p + x_K w_K$ → Gesamtpreis



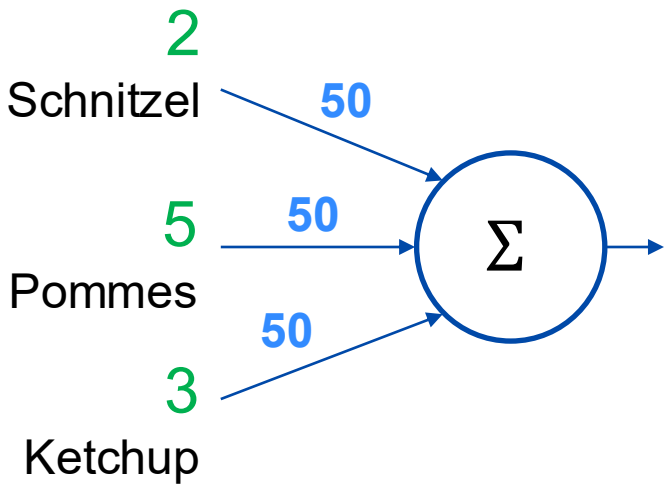
Beispiel: Mensa

- Bekannt: Anzahl der Portionen, Gesamtpreis
- Tatsächliche Gewichte



Beispiel: Mensa

- Bekannt: Anzahl der Portionen, Gesamtpreis
- Initiale Gewichte



Errechneter Preis:
500 = y

- $2 * 50 + 5 * 50 + 3 * 50 = 500$
- Delta-Regel:

$$\Delta w_i = -\varepsilon\, x_i\, (\hat{y} - y)$$

- $\hat{y} - y = 850 - 500 = 350$
- Lernrate $\varepsilon = \frac{1}{35}$

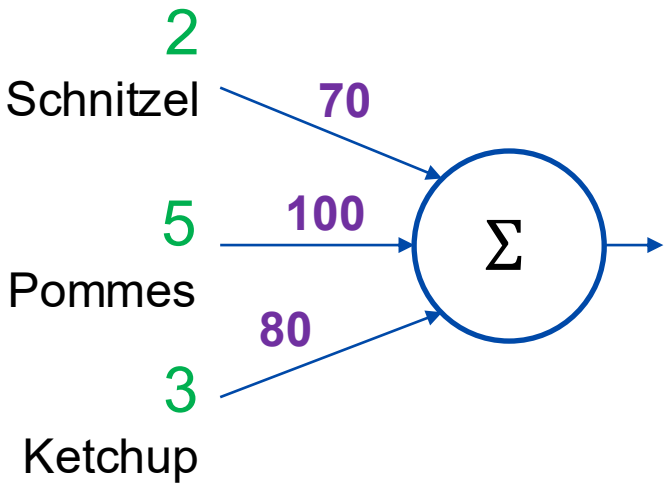
$$\Delta w_{Schnitzel} = -\frac{1}{35} * 2 * 350 = -20$$

$$\Delta w_{Pommes} = -\frac{1}{35} * 5 * 350 = -50$$

$$\Delta w_{Ketchup} = -\frac{1}{35} * 3 * 350 = -30$$

Beispiel: Mensa

- Bekannt: Anzahl der Portionen, Gesamtpreis
- Neue Gewichte



- $2 * 70 + 5 * 100 + 3 * 80 = 880$
- $\hat{y} - y = 850 - 880 = -30$
- Lernrate $\varepsilon = \frac{1}{35}$

$$\Delta w_{Schnitzel} = -\frac{1}{35} * 2 * -30 = 1.71$$

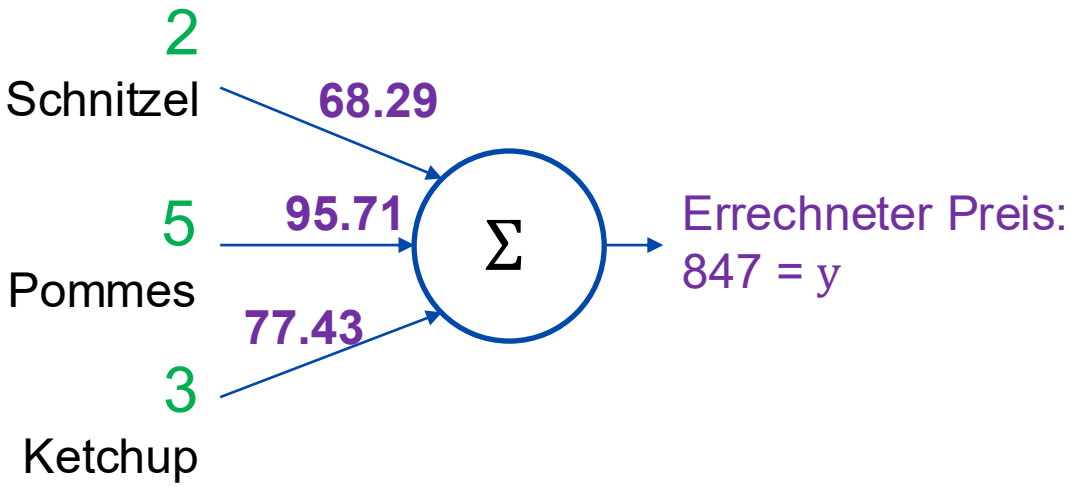
$$\Delta w_{Pommes} = -\frac{1}{35} * 5 * -30 = 4.29$$

$$\Delta w_{Ketchup} = -\frac{1}{35} * 3 * -30 = 2.57$$

Beispiel: Mensa

- Bekannt: Anzahl der Portionen, Gesamtpreis
- Neue Gewichte

- $2 * 68.29 + 5 * 95.71 + 3 * 77.43 = 847$
- $\hat{y} - y = 850 - 847 = 3$



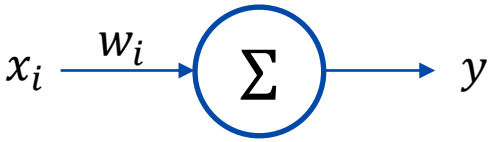
Wie kommen wir zur Delta-Regel

Durch Einsetzen unserer quadratischen Kostenfunktion:

$$C = \frac{1}{2}(\hat{y} - y)^2$$

Kettenregel:

$$\frac{\partial C}{\partial w_i} = \frac{dC}{dy}\ \frac{\partial y}{\partial w_i} = (-1)(\hat{y} - y)\, x_i$$



$$y = w_i x_i \qquad \frac{\partial y}{\partial w_i} = x_i$$

Delta-Regel:

$$\Delta w_i = -\varepsilon \frac{\partial C}{\partial w_i} = \varepsilon x_i (\hat{y} - y)$$

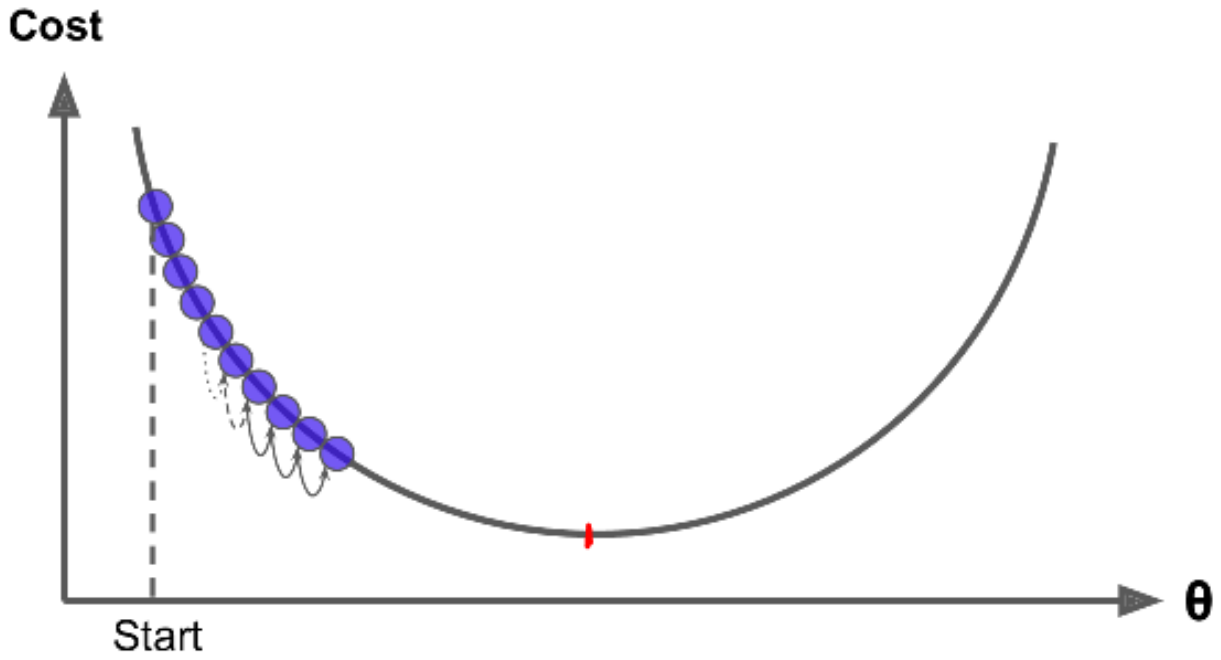
Pseudocode

- Eingabe
  - $\boldsymbol{X}$ **:** Matrix mit n Trainingsvektoren/Beispielen
  - Randomisiert initialisierte Werte für w_0 und w_1
  - predict(X): $\mathbf{y} = w_1\boldsymbol{X} + w_0$
  - learnRate
  - # of epochs

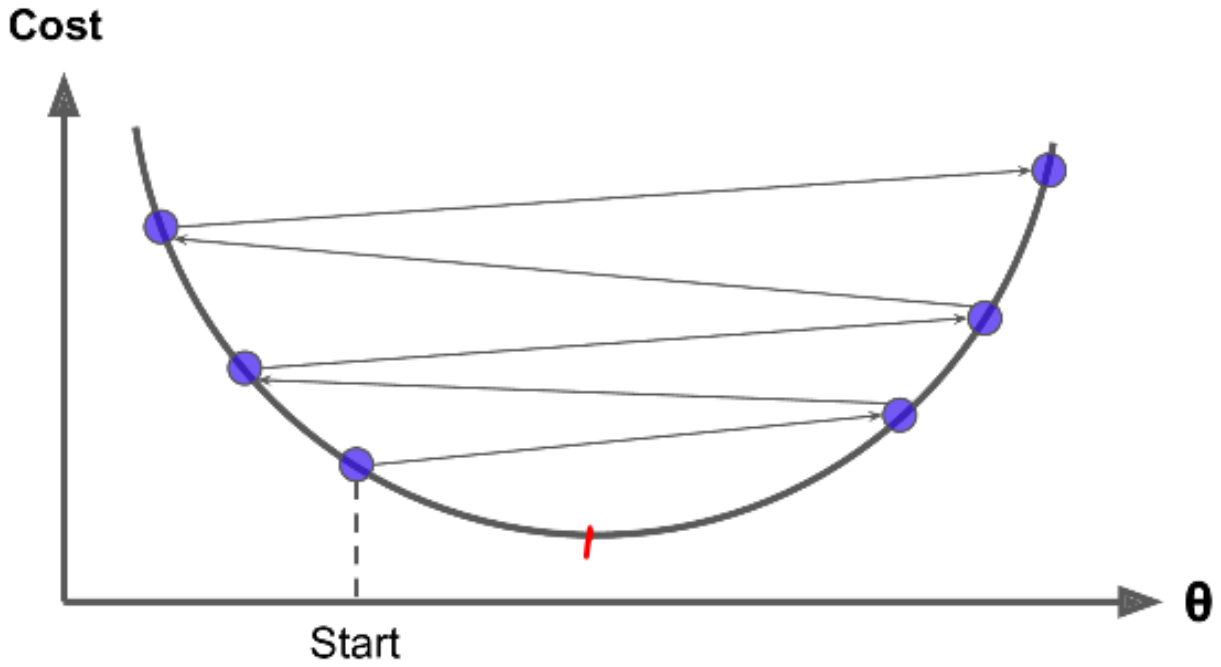
- Fit → For e in epochs:
  - $\mathbf{y_pred}$ = predict($\mathbf{X}$)
  - $delta_w_1$ = (-1/n) * sum($\boldsymbol{X} * (\mathbf{y} - \mathbf{y_pred})$) # Partielle Ableitung für w_1 → $\frac{\partial C}{\partial w_1}$
  - $delta_w_0$ = (-1/n) * sum($\mathbf{y} - \mathbf{y_pred}$) # Partielle Ableitung für w_0 → $\frac{\partial C}{\partial w_0}$
  - w_1 = w_1 - learnRate * $delta_w_1$ # Update w_1
  - w_0 = w_0 - learnRate * $delta_w_0$ # Update w_0

Das Problem mit der Lernrate beim Gradientenabstieg

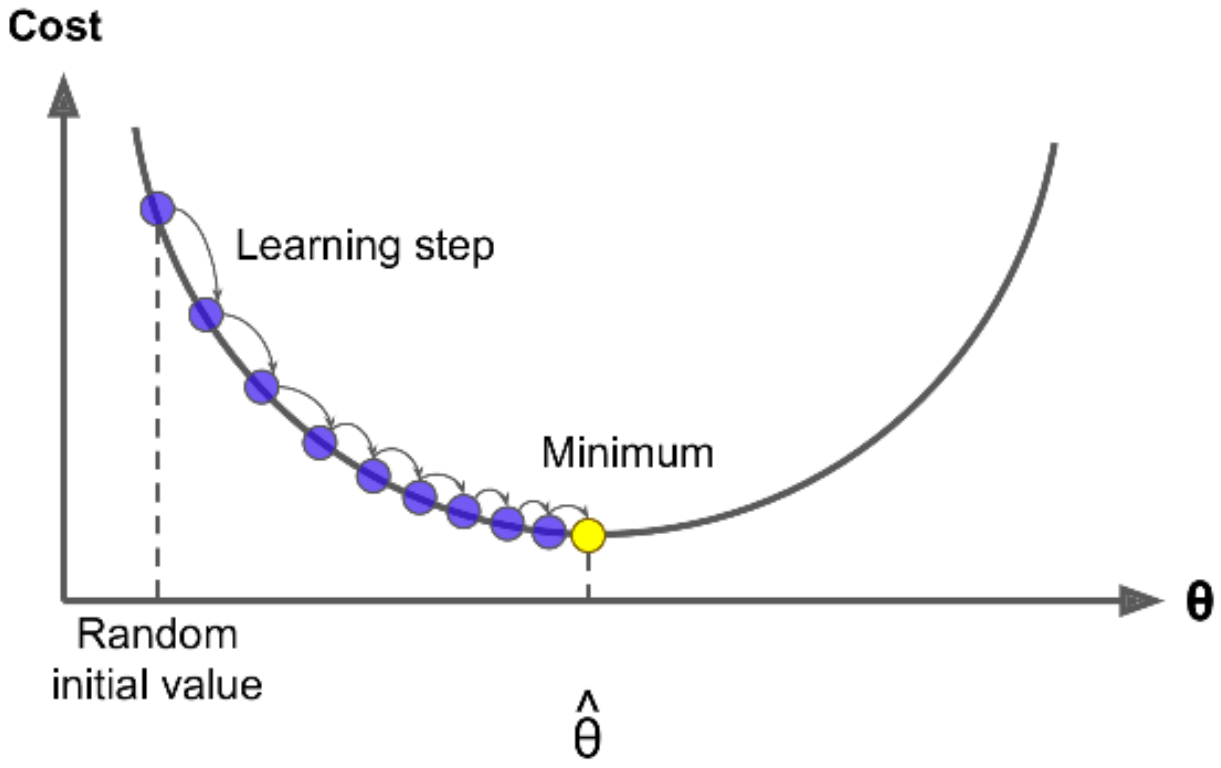
Lernrate – Zu gering



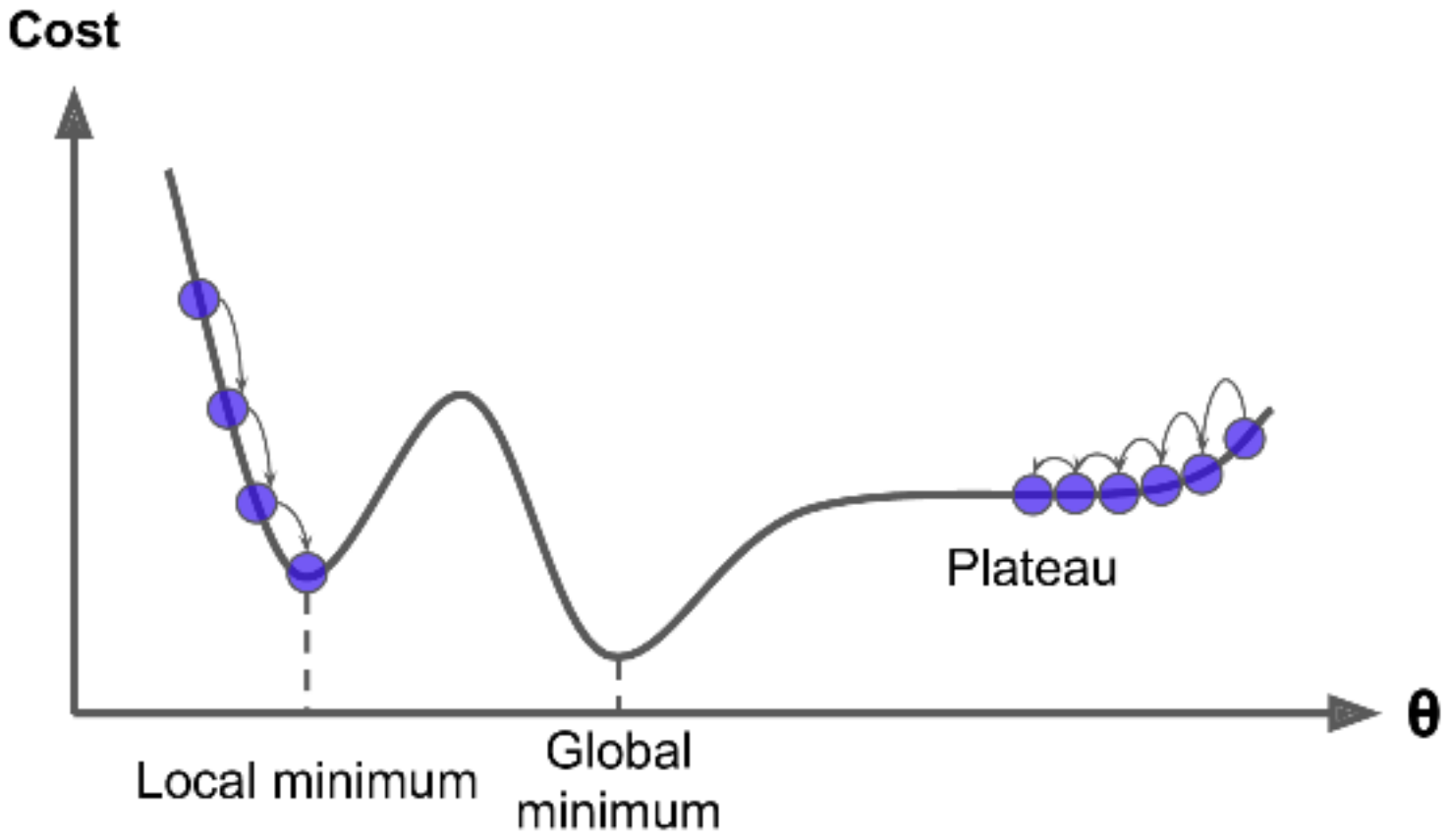
Lernrate – Zu hoch



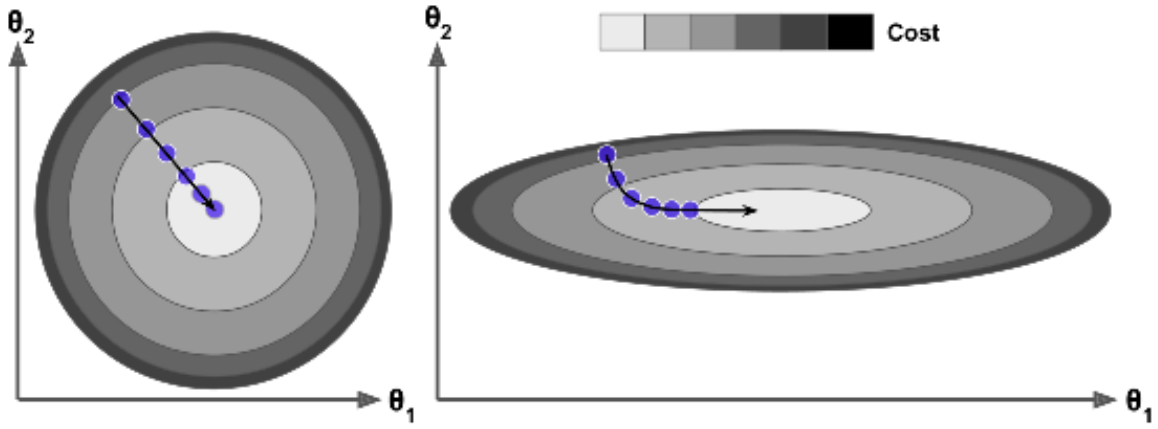
Lernrate – Ideal



Probleme



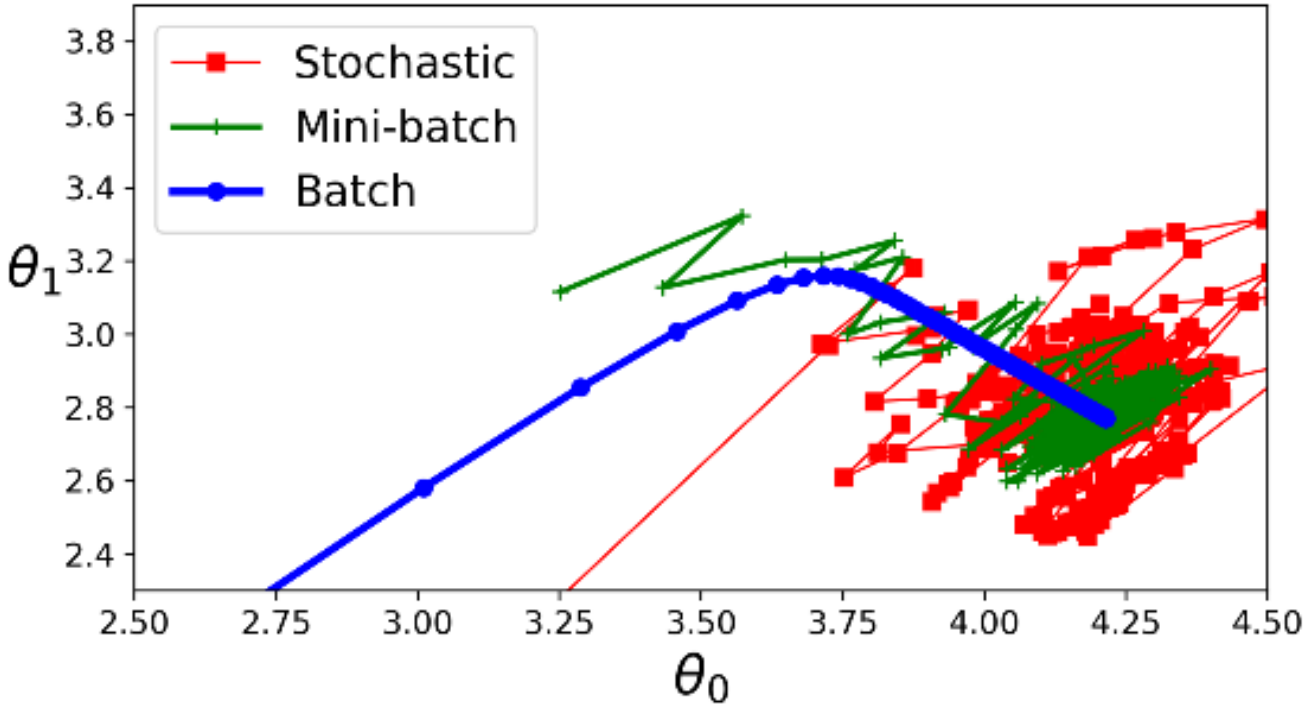
Probleme – Feature Scaling



Unterschiedliche Abstiegsverfahren

- Stochastic gradient descent
- Batch gradient descent
- Mini-batch gradient descent

Vergleich der Gradientenabstiegsarten



Güte der Regression bestimmen – Mean Square Error

$$\frac{1}{M}\sum_{i=1}^{M}\left(\hat{y}_i - f(x_i)\right)^2$$

Beispiel:

$\hat{y}$ = [3, -0.5, 2, 7]

$f(x)$ = [2.5, 0.0, 2, 8]

Güte der Regression bestimmen – Mean Square Error

$$\frac{1}{M}\sum_{i=1}^{M}\left(\hat{y}_i - f(x_i)\right)^2$$

Beispiel:

$\hat{y}$ = [3, -0.5, 2, 7]
$f(x)$ = [2.5, 0.0, 2, 8]

$$= \frac{1}{4}((3-2.5)^2 + (-0.5-0.0)^2 + (2-2)^2 + (7-8)^2)$$

$$= \frac{1}{4}(0.5^2 + (-0.5)^2 + 0^2 + (-1)^2)$$

$$= \frac{1}{4}(0.25 + 0.25 + 0 + 1) = \frac{1.5}{4} = 0.375$$

Güte der Regression bestimmen -- Korrelationskoeffizient

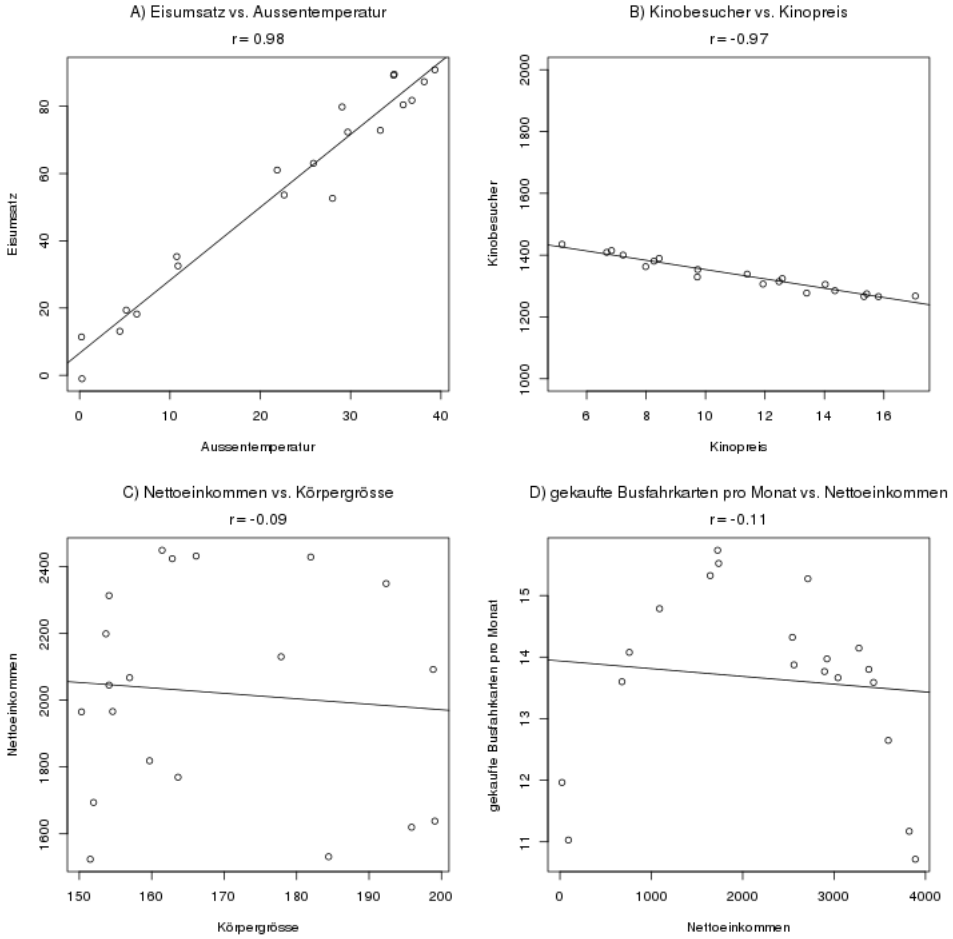
- Maß für den Grad des Linearen Zusammenhangs
- Kann positiv oder negativ sein
- +1 = vollständiger positiver Zusammenhang, -1 = vollständig negativer Zusammenhang
- Faustregeln:
  - 0 = kein linearer Zusammenhang
  - 0.3 = schwach positiver linearer Zusammenhang
  - 0.5 = mittelstarker positiver linearer Zusammenhang
  - 0.8 = starker positiver linearer Zusammenhang
  - -0.3 = schwach negativer linearer Zusammenhang
  - -0.5 = mittelstarker negativer linearer Zusammenhang
  - -0.8 = starker negativer linearer Zusammenhang

Güte der Regression bestimmen -- Korrelationskoeffizient

- Pearson-Korrelation: Standardisierte Kovarianz zweier Variablen
- Kovarianz: $Cov(x,y) = \frac{1}{n-1}\sum_{i=1}^{n}(x_i - \overline{x})(y_i - \overline{y})$
  - Kovarianz trifft Aussage über den Zusammenhang zweier Zufallsvariablen
  - Gehen hohe Werte der einen mit hohen Werten der anderen Zufallsvariable einher
  - Hängt von der Varianz von x und y ab
- Korrelation: $r_{xy} = \frac{Cov(x,y)}{s_x * s_y}$ $\quad r_{xy} := \frac{\frac{1}{n}\sum_{i=1}^{n}(x_i - \overline{x})(y_i - \overline{y})}{\sqrt{\frac{1}{n}\sum_{i=1}^{n}(x_i - \overline{x})^2} \cdot \sqrt{\frac{1}{n}\sum_{i=1}^{n}(y_i - \overline{y})^2}}.$

  = Kovarianz / Standardabweichung von x und y

Beispiele



https://www.crashkurs-statistik.de/der-korrelationskoeffizient-nach-pearson/

Evaluation

- Welche Evaluationsmechanismen hatten wir bei der Klassifikation besprochen?
  - Hold-out
  - Kreuzvalidierung
    - K-fold Cross-Validation
    - Stratified K-fold
    - Leave-One-Out Cross Validation

- Funktionieren auch für Regression

Zusammenfassung

- Einführung in die Lineare Regression
- Wozu wird die Lineare Regression verwendet?
- Lernverfahren:
  - Analytisch: <Name>?
  - Iterativ: <Name>?
- Multiple Lineare Regression
- Gütebestimmung der Regression